

Module 6 Activity

June 28, 2026

Unnas Hussain

1 Problem 1: Linear Programming

1.1 Objective:

Solve a linear programming (LP) problem using Python, analyze the results, and interpret the solution in a real-world context.

1.2 Code:

```
[8]: from scipy.optimize import linprog

# Coefficients of the objective function (profit) (negative for maximization)
# c = [-3, -5]
c = [-3, -6]

# Coefficients for the inequality constraints (A_ub @ x <= b_ub)
# Constraint 1: 1*A + 2*B <= 8 (time constraint)
# Constraint 2: 3*A + 2*B <= 12 (resource constraint)
A = [[1, 2], [3, 2]]
b = [8, 12]

# Bounds for variables (non-negativity: x >= 0)
x_bounds = [(0, None), (0, None)]

# Solve the linear programming problem
result = linprog(c, A_ub=A, b_ub=b, bounds=x_bounds, method='highs')

# Output results
print("Optimal Value (Max Profit):", -result.fun) # Flip the sign for
↳ maximization
print("Optimal Solution:", result.x)
print(f"Product A: {result.x[0]} units")
print(f"Product B: {result.x[1]} units")
```

Optimal Value (Max Profit): 24.0
Optimal Solution: [0. 4.]
Product A: 0.0 units
Product B: 4.0 units

1.3 Questions:

1. Update the code to reflect the problem statement and solve the linear programming problem. What changes did you make to the code?
 - Set the object function `c` to reflect the profits for A and B.
 - Set constraints matrix `A` according to the time and resource constraints
 - Set constraint bounds `b` for the amount of time and resources available
 - I also updated the linear programming method to `highs` because `simplex` is depreciated
2. What do the optimal values for Product (A) and Product (B) represent?
 - 2 units of A and 3 units of B will net \$21
3. How would the solution change if the profit for Product (B) increased to \$6?
 - 0 units of A and 4 units of B will net \$24
4. How does the feasible region influence the optimal solution?
 - The feasible region is the set of all (A, B) combinations satisfying all constraints. It forms a polygon on the x y plane bounded by the following set of linear inequalities (representing the constraints):
 - $x \geq 0, y \geq 0$ (non-negativity)
 - $x + 2y \leq 8$ (time)
 - $3x + 2y \leq 12$ (resources)
 - The optimal solution always occurs at a vertex (corner point) of this feasible region. The `simplex` algorithm moves along edges from vertex to vertex, improving the objective until reaching the optimal corner. In this case, the optimal vertex is at (2, 3), where both the time and resource constraints are binding (both are equalities at this point). The `highs` solver is a more efficient way to navigate the interior of the region, and then switches to `simplex` when it is optimal to do so.

2 Problem 2: Quadratic Programming

2.1 Objective:

Solve a quadratic programming (QP) problem by coding the objective function and constraints into CVXOPT and analyzing the solution.

2.2 Code:

```
[9]: from cvxopt import matrix, solvers

# Define Q (quadratic term) and c (linear term)
# Objective: Minimize  $2x_1^2 - x_1x_2 + 4x_2^2 - 3x_1 - 2x_2$ 
# CVXOPT form:  $(1/2)x^T Q x + c^T x$ , so double the quadratic coefficients
Q = matrix([[4.0, -1.0], [-1.0, 8.0]])
c = matrix([-3.0, -2.0])

# Inequality constraints  $Gx \leq h$ 
G = matrix([[-1.0, 0.0], [0.0, -1.0]])
h = matrix([0.0, 0.0])

# Equality constraints  $Ax = b$ 
A = matrix([[1.0], [1.0]])
b = matrix([1.0])

# Solve the QP problem
solvers.options['show_progress'] = False
sol = solvers.qp(Q, c, G, h, A, b)

print("Optimal Solution:", sol['x'])
print("Optimal Value:", sol['primal objective'])
```

```
Optimal Solution: [ 7.14e-01]
[ 2.86e-01]
```

```
Optimal Value: -1.5714285714285714
```

2.3 Questions:

1. What are the optimal values for x_1 and x_2 , and what do they represent in the context of the problem?
 - $x_1 = 0.714$, $x_2 = 0.286$. Allocate ~71% to Project 1 and ~29% to Project 2 to minimize cost. The minimum cost is -1.57 (negative indicates net benefit from the linear reward terms).
2. How does the constraint $x_1 + x_2 = 1$ impact the solution space?
 - It restricts the 2D search space to a 1D line segment between (0,1) and (1,0). All resources must be fully allocated.
3. If the constraint $x_1 + x_2 = 1$ were replaced with $x_1 + x_2 \leq 1$, how would the solution change?
 - The solution would shift since the optimizer gains freedom to allocate less than 100% total if that reduces cost.

3 Problem 3: Analyzing the Simplex Method

3.1 Objective:

Analyze the provided Simplex algorithm implementation by stepping through the code, interpreting its operations, and understanding its runtime complexity.

3.2 Code:

```
[10]: import numpy as np
import pandas as pd

def simplex_algorithm(c, A, b):
    m, n = A.shape # m=constraints, n=variables --> O(1)

    # Add slack variables (identity matrix) to convert <= to = constraints --> O(m^2)
    slack_vars = np.eye(m)
    # Build initial tableau: [A | I | b] where I is slack vars --> O(m*(n+m))
    tableau = np.hstack([A, slack_vars, b.reshape(-1, 1)])

    # Objective row: negate c for maximization, pad with zeros --> O(n+m)
    obj_row = np.hstack([-c, np.zeros(m + 1)])
    # Stack objective row below constraints to complete tableau --> O(m*(n+m))
    tableau = np.vstack([tableau, obj_row])

    # Track which variables are basic (initially slack vars) --> O(m)
    basic_vars = [n + i for i in range(m)]
    non_basic_vars = list(range(n)) # O(n)

    step = 0

    while True: # Iterates T times; typically O(m), worst case exponential
        print(f"Step {step}: Tableau")
        df = pd.DataFrame(tableau,
            columns=[f"x{i+1}" for i in range(n+m)] + ["RHS"],
            index=[f"Constraint {i+1}" for i in range(m)] + ["Objective"])
        print(df, "\n")

        # Optimal when no negative coefficients in objective row --> O(n+m)
        if np.all(tableau[-1, :-1] >= 0):
            print("Optimal solution found!\n")
            break

        # Entering var: most negative coeff in objective row (biggest improvement) --> O(n+m)
```

```

entering = np.argmin(tableau[-1, :-1])

# Leaving var: smallest positive ratio RHS/pivot_col (tightest
↳constraint) --> O(m)
ratios = [tableau[i, -1] / tableau[i, entering] if tableau[i, entering]
↳> 0 else np.inf for i in range(m)]
leaving = np.argmin(ratios)

# Unbounded if no positive ratio exists
if ratios[leaving] == np.inf:
    raise ValueError("Problem is unbounded!")

# Pivot step 1: normalize pivot row so pivot element becomes 1 -->
↳O(n+m)
pivot = tableau[leaving, entering]
tableau[leaving, :] /= pivot

# Pivot step 2: eliminate entering var from all other rows -->
↳O(m*(n+m))
for i in range(m + 1):
    if i != leaving:
        tableau[i, :] -= tableau[i, entering] * tableau[leaving, :]

# Swap entering var into basis, replacing leaving var --> O(1)
basic_vars[leaving] = entering
step += 1

# Extract solution: basic vars get their RHS values, others are 0 --> O(m)
solution = np.zeros(n + m)
for i, var in enumerate(basic_vars):
    if var < n:
        solution[var] = tableau[i, -1]

print("Optimal Value:", tableau[-1, -1])
print("Solution:", solution[:n])
return tableau[-1, -1], solution[:n]

c = np.array([3, 5])
A = np.array([[1, 2], [3, 2]])
b = np.array([8, 12])
simplex_algorithm(c, A, b)

```

Step 0: Tableau

	x1	x2	x3	x4	RHS
Constraint 1	1.0	2.0	1.0	0.0	8.0
Constraint 2	3.0	2.0	0.0	1.0	12.0
Objective	-3.0	-5.0	0.0	0.0	0.0

Step 1: Tableau

	x1	x2	x3	x4	RHS
Constraint 1	0.5	1.0	0.5	0.0	4.0
Constraint 2	2.0	0.0	-1.0	1.0	4.0
Objective	-0.5	0.0	2.5	0.0	20.0

Step 2: Tableau

	x1	x2	x3	x4	RHS
Constraint 1	0.0	1.0	0.75	-0.25	3.0
Constraint 2	1.0	0.0	-0.50	0.50	2.0
Objective	0.0	0.0	2.25	0.25	21.0

Optimal solution found!

Optimal Value: 21.0

Solution: [2. 3.]

```
[10]: (np.float64(21.0), array([2., 3.]))
```

3.2.1 Runtime summary table:

- Setup (slack vars, tableau): $O(m*(n+m))$
- Optimality check: $O(n+m)$
- Find entering/leaving: $O(n+m)$
- Pivot operation: $O(m*(n+m))$
- Per iteration: $O(m*(n+m))$
- Total (typical): $O(m^2 * n)$